

代码覆盖率及路径分析检测

软件测试课程大作业

徐宇鸿

计算机科学与工程学院

2026 年 4 月 19 日



1 什么是路径分析与覆盖率

2 核心概念与工作原理

3 PathTracer 工具实现

4 使用演示

5 总结





程序执行路径与三种基本结构

程序运行时并非所有代码都会被执行——**条件分支**和**循环**决定了程序走哪条路。**路径分析**就是追踪程序实际走了哪些路，找出哪些路从未走过。

顺序结构

代码从上到下依次执行，**只有一条路径**，不存在分支。

```
x = 1
y = x + 2
print(y)
```

选择结构（分支）

根据条件走**不同分支**，产生**多条路径**。

```
if x > 0:
    return " 正"
elif x == 0:
    return " 零"
else:
    return " 负"
```

循环结构

重复执行某段代码，循环次数影响路径。

```
for i in range(n):
    total += i
while x > 0:
    x = x // 2
```

分支覆盖率衡量的是：程序中所有分支点里，有多少被测试执行过。

$$\text{分支覆盖率} = \frac{\text{已执行的分支数}}{\text{总分支数}} \times 100\%$$

例：程序有 5 个分支（3 个 if + 1 个 for + 1 个 try），运行后走了 3 个 $\Rightarrow$ 覆盖率 60%。
覆盖率越高 $\Rightarrow$ 测试越充分，隐藏 bug 越少。



路径分析与软件测试的关系

路径分析是白盒测试的核心手段——通过追踪程序内部执行路径，量化测试的**充分性**。

测试流程：

- 1 **静态分析**：解析源码，找出所有分支点 (if/for/while/try)
- 2 **设计测试用例**：构造不同输入，尽可能覆盖每条分支
- 3 **动态追踪**：运行程序，记录实际走了哪些行/分支
- 4 **计算覆盖率**：对比静态分支与动态执行结果
- 5 **补充测试**：对未覆盖分支补充测试用例，提升覆盖率

本作业目标：编写 PathTracer 工具，自动完成上述步骤 1-4，生成描述程序执行分支的覆盖率报告。

为什么要做路径分析？

如果不追踪执行路径，你**无法知道**某段 else 分支是否被测试过。

未覆盖 = 未测试 = 可能藏有 bug

典型场景：

- 异常处理 (except) 从未触发
- 边界条件 (elif) 从未命中
- 循环体从未进入或从未跳过

路径分析工具的工作分为三个阶段：

- 1 **静态分析**——不运行程序，用 Python 的 `ast` 模块解析源码，找出所有分支点（`if` / `for` / `while` / `try`）
- 2 **动态追踪**——运行程序，用 `sys.settrace()` 在虚拟机级别钩住每一步执行，记录实际走了哪些行
- 3 **覆盖率计算**——对比两份结果：分支点所在行是否出现在已执行行号集合中，得到覆盖率百分比





静态分析 (1/2): AST——抽象语法树是什么?

Python 执行代码前会先把源码经过**词法分析** → **语法分析** → 变成 AST (一棵树), 再由虚拟机执行。ast 模块让我们能直接访问这棵树。

一段简单的源码

```
def f(x):  
    if x > 0:  
        return 1  
    else:  
        return -1
```

对应的 AST 树形结构

Module

```
↪ FunctionDef(name='f')  
    ↪ If(test=Compare(x>0))  
        ↪ Return(Constant(1))  
        ↪ orelse:  
            ↪
```

Return(Constant(-1))

每个 if/for/while/try 在树中都有**对应的节点类型**: ast.If、ast.For……

关键点: 我们不需要自己写解析器——Python 标准库 `ast.parse()` 一行调用就能把任意源码变成这棵树。然后用 `ast.walk(tree)` 深度优先遍历所有节点, 逐个检查类型即可。

静态分析 (2/2): 项目中的实现——cfg_builder.py

前一页讲了 AST 的原理。现在来看我们项目中**实际写的代码**:

cfg_builder.py 用 ast 模块遍历语法树, 识别每种控制流节点, 提取分支信息。

```
class CFGBuilder:
    def build_from_file(self, file_path):
        source = open(file_path).read()
        tree = ast.parse(source)                # 源码 -> AST 树
        for node in ast.walk(tree):            # 遍历所有节点
            branch = self._extract_branch(node)
            if branch:
                self.branches.append(branch)

    def _extract_branch(self, node):
        if isinstance(node, ast.If):
            return CodeBranch(line_no=node.lineno,
                               branch_type=BranchType.IF, end_line=node.end_lineno)
        elif isinstance(node, ast.For):
            return CodeBranch(..., BranchType.FOR)
        elif isinstance(node, ast.While):
            return CodeBranch(..., BranchType.WHILE)
        elif isinstance(node, ast.ExceptHandler):
            return CodeBranch(..., BranchType.EXCEPT)
        return None    # 其他节点不产生分支
```



前置知识 (1/2): 源码 → 字节码

要理解动态追踪的原理, 需要先了解 Python 代码的执行过程。

Python 不会直接执行你写的代码

它先把源码**编译**成“字节码”(Bytecode)——一种简化的中间指令。源码的一行可能对应多条字节码指令。

可以用 Python 内置的 `dis` 模块查看:

你写的源码

```
x = 1 + 2
```

翻译后的字节码

LOAD_CONST	1	# 取出数字 1
LOAD_CONST	2	# 取出数字 2
BINARY_ADD		# 做加法
STORE_NAME	x	# 存到变量 x

类比: 源码像“中文”, 字节码像“英文”, 虚拟机能读懂英文并逐条执行。我们不需要手写字节码——Python 自动完成翻译。





前置知识 (2/2): 虚拟机与栈帧

CPython 虚拟机 (Virtual Machine) 是一个用 C 语言写的**大循环**，它逐条读取字节码并执行。

栈帧 (Frame) —— 虚拟机的“笔记本”

每条字节码执行时，虚拟机维护一个**栈帧**，里面记录着当前执行状态：

- `f_lineno` —— 当前执行到源码**第几行**
- `f_locals` —— 当前所有**局部变量**的值
- `f_code.co_name` —— 当前在**哪个函数里**
- `f_code.co_varnames` —— 函数的**参数名列表**

调用栈 (Call Stack) —— 函数嵌套的“俄罗斯套娃”

当函数 A 调用函数 B 时，虚拟机会创建一个新的栈帧压入**调用栈**。函数返回时弹出。

调用栈让虚拟机知道：“执行完后该回到哪里继续”。

关键：`sys.settrace()` 就是让虚拟机在每条字节码执行前后“暂停一下”，把当前栈帧交给你检查。这样你就能知道程序执行到了哪一行、变量是什么值。

动态追踪 (1/2): `sys.settrace()` 的工作机制

`sys.settrace()` 是 CPython 虚拟机提供的**调试钩子**——不需要修改被追踪程序的源码，从“外部”监控执行。

底层原理

CPython 的主执行循环 (`Python/ceval.c`) 在每条**字节码**执行前后，都会检查是否设置了 `trace function` (即你通过 `sys.settrace()` (回调函数) 注册的那个函数)。如果有，就暂停执行，把**栈帧**传给你的回调函数。这是**解释器级别**的钩子——不需要修改被追踪的代码。

回调函数接收的三个参数

参数	含义
<code>frame</code>	当前 栈帧对象 : <code>f_lineno</code> (行号)、 <code>f_code.co_name</code> (函数名)、 <code>f_locals</code> (局部变量)
<code>event</code>	事件类型: <code>'call'</code> (函数调用)、 <code>'line'</code> (执行一行)、 <code>'return'</code> (返回)、 <code>'exception'</code> (异常)
<code>arg</code>	附加数据: <code>'return'</code> → 返回值; <code>'exception'</code> → (<code>exc_type</code> , <code>exc_value</code> , <code>traceback</code>)





动态追踪 (2/2): 项目中的实现——tracer.py

前一页讲了 `sys.settrace()` 的原理。现在来看我们项目中**实际写的代码**:
`tracer.py` 里的 `_trace_callback` 方法就是注册给 `sys.settrace()` 的那个回调函数。
它根据不同事件分别处理: 记录行号、函数调用链和返回值。

```
def _trace_callback(self, frame, event, arg):
    if not frame.f_code.co_filename.endswith(self._target_file):
        return self._trace_callback          # 只追踪目标文件

    if event == 'line':
        self.executed_lines.add(frame.f_lineno)          # 记录行号

    elif event == 'call':
        args = {k: repr(frame.f_locals[k])}              # 提取参数
            for k in frame.f_code.co_varnames}
        call = FunctionCall(function_name=frame.f_code.co_name,
                             call_depth=len(self._call_stack), arguments=args)
        if self._call_stack:
            self._call_stack[-1].children.append(call)   # 子调用
        else:
            self.function_calls.append(call)              # 顶层调用
            self._call_stack.append(call)                 # 入栈

    elif event == 'return':
        call = self._call_stack.pop()                    # 出栈
        call.return_value = repr(arg)                    # 记录返回值
```

要点: `_call_stack` 列表模拟函数调用栈 → 还原完整调用树 (谁调用谁、参数、返回值)。

reporter.py 将静态分析的分支列表与动态追踪的行号集合求交集，得到覆盖情况。

判定逻辑（逐分支检查）

- 1 cfg_builder 解析源码 → 得到所有 CodeBranch 列表，每个记录 line_no 和 branch_type
- 2 tracer 运行程序 → 得到 executed_lines: Set[int]（已执行行号集合）
- 3 对每个分支：
if branch.line_no in executed_lines:
branch.is_executed = True
- 4 统计覆盖率：

$$\text{coverage} = \frac{\sum 1[\text{branch.is_executed}]}{|\text{branches}|} \times 100\%$$

- 5 按类型分组统计：if / for / while / try / except 各多少

具体示例

静态分析结果（10 个分支）：

Line 2: if ✓ Line 4: if ✓ Line 9: for ✓
Line 14: if ✓ Line 18: while ✓ Line 22: try ✓
Line 25: except × Line 28: for ✓
Line 32: if × Line 35: while ×

已执行 7/10，覆盖率 70%

未覆盖：except（没触发异常）、if（条件未命中）、while（循环未进入）

启示：需要构造能触发异常、命中特定条件、进入循环的测试用例。





文件列表

文件	职责
models.py	数据模型定义
cfg_builder.py	静态分析 (CFG)
tracer.py	动态追踪
reporter.py	覆盖率计算与报告
cli.py	命令行入口
demo.py	演示脚本

模块协作流程

- 1 `cfg_builder` 解析源代码
↓ 得到所有分支点
- 2 `tracer` 追踪运行过程
↓ 得到已执行行号
- 3 `reporter` 对比 → 覆盖率
↓
- 4 输出报告 (文本/HTML/JSON)



数据模型 (1/2): 分支类型与分支点

models.py 定义了整个工具使用的数据结构。涉及两个 Python 基础概念:

```
class BranchType(Enum):  
    """ 所有可识别的分支类型 """  
    SEQUENTIAL = "sequential"  
    IF          = "if"  
    ELSE       = "else"  
    FOR        = "for"  
    WHILE      = "while"  
    TRY        = "try"  
    EXCEPT   = "except"
```

```
@dataclass  
class CodeBranch:  
    """ 表示代码中的一个分支点 """  
    file_path: str  
    line_no: int  
    branch_type: BranchType  
    end_line: int = 0  
    is_executed: bool = False
```

Enum 枚举是什么?

枚举就是“一组有名字的常量”。

用数字记分支类型容易混淆 (1=if? 2=for?),
用 BranchType.IF 这样的名字就清晰多了。

Python 标准库 enum.Enum 提供这个功能。

@dataclass 装饰器是什么?

普通 class 要自己写 `__init__` 方法来初始化属性。`@dataclass` 让 Python **自动生成**这些代码, 你只需声明字段即可。

相当于自动帮你生成了:

```
def __init__(self, file_path,  
             line_no, ...):  
    self.file_path = file_path  
    self.line_no = line_no  
    ...
```

数据模型 (2/2): 函数调用与执行路径

还需要记录**函数调用链**和**整体执行路径**:

```
@dataclass
class FunctionCall:
    """记录一次函数调用"""
    function_name: str          # 函数名
    file_path: str              # 所在文件
    line_no: int                # 调用位置行号
    call_depth: int             # 调用深度 (0=顶层调用)
    arguments: Dict[str, str] = field(...) # 参数名 -> 参数值 (字符串)
    return_value: Optional[str] = None     # 返回值 (字符串)
    children: List[FunctionCall] = field(...) # 子调用列表 (嵌套结构)

@dataclass
class ExecutionPath:
    """记录一个文件的完整执行路径"""
    file_path: str
    executed_lines: Set[int] = field(...) # 执行过的所有行号
    executed_branches: Dict[int, CodeBranch] = field(...) # 被执行的分支
    function_calls: List[FunctionCall] = field(...) # 函数调用链

@dataclass
class CoverageReport:
    """最终的覆盖率报告"""
    total_branches: int = 0
    executed_branches: int = 0
    coverage_percentage: float = 0.0
    branches_by_type: Dict[BranchType, int] = field(...) # 按类型统计
    file_reports: Dict[str, ExecutionPath] = field(...) # 按文件统计
```





被追踪的示例代码 (1/2): 条件分支

demo.py 包含多种程序结构, 演示 PathTracer 的追踪能力:

if/elif/else 选择结构

```
def demo_conditionals(value):  
    if value < 0:  
        return "negative"  
    elif value == 0:  
        return "zero"  
    elif value <= 10:  
        return "small"  
    elif value <= 50:  
        return "medium"  
    else:  
        return "large"
```

说明

5 条可能的执行路径, 输入不同走的路不同:

- demo_conditionals(-5)
走第 1 个 if → "negative"
- demo_conditionals(25)
走第 4 个 elif → "medium"
- demo_conditionals(100)
走 else → "large"

单一输入**只能覆盖一条路径**。



被追踪的示例代码 (2/2): 循环、异常与嵌套

除了 if/else, demo.py 还包含循环、异常处理和嵌套结构:

for 循环

```
def demo_for_loop(items):  
    total = 0  
    for item in items:  
        total += item  
    return total
```

while 循环

```
def demo_while_loop(limit):  
    results = []  
    counter = 0  
    while counter < limit:  
        results.append(counter * 2)  
        counter += 1  
    return results
```

try/except 异常处理

```
def demo_exception_handling(  
    dividend, divisor):  
    try:  
        result = dividend / divisor  
    except ZeroDivisionError:  
        result = 0.0  
    else:  
        result = result * 1.0  
    finally:  
        pass  
    return result
```

嵌套结构 (循环 + 条件)

```
def demo_nested(data, threshold):  
    results = []  
    if threshold > 0:  
        for item in data:
```

一行命令即可追踪任意 Python 程序的执行路径，**不需要修改被追踪的代码：**

追踪 demo.py，输出文本报告到终端

```
python -m pathtracer.cli pathtracer/demo.py
```

输出 HTML 格式的覆盖率报告（源码逐行高亮）

```
python -m pathtracer.cli --format html -o report.html pathtracer/demo.py
```

输出 JSON 格式（方便后续程序处理）

```
python -m pathtracer.cli --format json -o report.json pathtracer/demo.py
```

静默模式：只输出报告，不显示程序自身的打印

```
python -m pathtracer.cli --quiet pathtracer/demo.py
```

原理：cli.py 用 compile() + exec() 在追踪环境下运行目标程序，追踪器自动记录一切。





运行结果：文本报告

运行 demo.py 后生成的覆盖率报告：

```
=====
EXECUTION PATH COVERAGE REPORT
=====
```

```
Total Branches: 15
Executed Branches: 12
Coverage: 80.0%
```

Branches by Type:

```
if      : 6
for     : 4
while   : 3
try     : 1
except  : 1
```

```
=====
EXECUTED BRANCHES:
-----
```

```
demo.py:
Line 32: if      [executed]
Line 34: if      [executed]
Line 44: for     [executed]
```

不同测试输入的覆盖对比

以 demo_conditionals 函数为例，不同输入覆盖不同分支：

测试输入	覆盖分支	未覆盖	覆盖率
demo_conditionals(25)	4/5	else (value>50)	80%
demo_conditionals(-5)	1/5	4 条路径	20%
demo_conditionals(0)	2/5	3 条路径	40%
综合多次调用	5/5	无	100%

结论

单一测试输入只能覆盖部分路径。要达到高覆盖率，需要**设计多组不同的测试输入**，覆盖所有分支。

这正体现了路径分析在软件测试中的价值：**量化测试充分性**，指导测试用例的补充。



做了什么

编写了 PathTracer 工具，能够自动追踪 Python 程序的执行路径并生成覆盖率报告。

核心技术

- 1 静态分析：用 ast 模块解析源代码，识别所有分支点
- 2 动态追踪：用 `sys.settrace()` 在运行时记录执行路径
- 3 覆盖率计算：对比静态分支与动态执行结果，计算覆盖率百分比

输出成果

- 文本报告（终端直接查看）
- HTML 可视化报告（源码逐行高亮 + 函数调用树）
- JSON 格式（供自动化工具使用）



结构类型	关键词	支持情况
顺序结构	逐行执行	✓ 行级追踪
选择结构	if / elif / else	✓ 分支覆盖
循环结构	for / while	✓ 进入/跳过
异常处理	try / except	✓ 正常/异常路径
函数调用	def / return	✓ 调用链 + 参数
嵌套结构	组合使用	✓ 递归追踪



谢谢！

